

SmartCampusManager

Database Relations & Schema Reference

Oracle XE · campus_admin schema · localhost:1522/xe

Course: SEng2022 — Design and Analysis of Data Structures and Algorithms
Debre Berhan University

1. Schema Overview

The database consists of **7 tables**, **4 sequences**, and **2 stored procedures**. All primary keys are generated by Oracle sequences (auto-increment). Foreign keys enforce referential integrity across all modules.

Table	Purpose	PK Sequence
USERS	All system users (Admin, Technician, Security, Auditor, Student)	SEQ_USERS
ASSETS	University physical assets (equipment, furniture)	SEQ_ASSETS
MAINTENANCE_LOG	Repair/maintenance requests linked to assets	—
STUDENT_ITEMS	Personal items registered by students	SEQ_STUDENT_ITEMS
LOST_FOUND	Lost or found item reports	SEQ_LOST_FOUND
LOCATIONS	Campus buildings and rooms	—
AUDIT_LOG	System-level event trail	—

2. Entity-Relationship Summary

Parent Table	Relationship	Child Table	FK Column
USERS	1 ■■■■ M	ASSETS	ASSETS.CREATED_BY
USERS	1 ■■■■ M	MAINTENANCE_LOG	MAINTENANCE_LOG.REPORTED_BY
USERS	1 ■■■■ M	STUDENT_ITEMS	STUDENT_ITEMS.STUDENT_ID
USERS	1 ■■■■ M	LOST_FOUND	LOST_FOUND.REPORTED_BY
ASSETS	1 ■■■■ M	MAINTENANCE_LOG	MAINTENANCE_LOG.ASSET_ID
LOCATIONS	1 ■■■■ M	STUDENT_ITEMS	STUDENT_ITEMS.LOCATION_ID
LOCATIONS	1 ■■■■ M	LOST_FOUND	LOST_FOUND.FOUND_LOCATION

3. Table Definitions

TABLE: USERS (PK sequence: SEQ_USERS)		
Column	Type	Constraint / Notes
USER_ID	NUMBER	PRIMARY KEY — auto from SEQ_USERS
USERNAME	VARCHAR2(50)	UNIQUE, NOT NULL
FULL_NAME	VARCHAR2(100)	NOT NULL
PASSWORD_HASH	VARCHAR2(64)	SHA-256 hex digest
ROLE	VARCHAR2(20)	ADMIN TECHNICIAN SECURITY AUDITOR STUDENT
EMAIL	VARCHAR2(100)	nullable
PHONE	VARCHAR2(20)	nullable
DEPARTMENT	VARCHAR2(100)	nullable
STUDENT_ID	VARCHAR2(30)	university student ID (STUDENT role only)
IS_ACTIVE	CHAR(1)	'Y' active / 'N' deactivated
CREATED_AT	DATE	DEFAULT SYSDATE

■ Central auth table — all FK references to a user point here.

TABLE: ASSETS (PK sequence: SEQ_ASSETS)		
Column	Type	Constraint / Notes
ASSET_ID	NUMBER	PRIMARY KEY — auto from SEQ_ASSETS
ASSET_NAME	VARCHAR2(100)	NOT NULL
CATEGORY	VARCHAR2(50)	e.g. Furniture, Electronics, Vehicle
SERIAL_NUMBER	VARCHAR2(100)	nullable — indexed in DSA BST
PURCHASE_VALUE	NUMBER(12, 2)	asset monetary value
CONDITION	VARCHAR2(30)	GOOD BROKEN UNDER_MAINTENANCE DECOMMISSIONED
CREATED_BY	NUMBER	FK → USERS.USER_ID

■ Sorted by Bubble Sort (by ID/value) and Merge Sort (by name/value) in AssetWindow.

TABLE: MAINTENANCE_LOG (PK sequence: — (own PK))		
Column	Type	Constraint / Notes
MAINTENANCE_ID	NUMBER	PRIMARY KEY
ASSET_ID	NUMBER	FK → ASSETS.ASSET_ID (NOT NULL)
REPORTED_BY	NUMBER	FK → USERS.USER_ID
DESCRIPTION	VARCHAR2(500)	issue description
STATUS	VARCHAR2(20)	PENDING IN_PROGRESS RESOLVED
PRIORITY	VARCHAR2(10)	URGENT HIGH NORMAL LOW

■ Greedy algorithm scores rows: URGENT=40, HIGH=30, NORMAL=20, LOW=10 + status bonus.

TABLE: LOCATIONS (PK sequence: — (own PK))		
Column	Type	Constraint / Notes
LOCATION_ID	NUMBER	PRIMARY KEY
BUILDING_NAME	VARCHAR2(100)	e.g. Library, Admin Block
ROOM_NUMBER	VARCHAR2(20)	e.g. 101, Lab-3

■ Shared lookup table used by STUDENT_ITEMS and LOST_FOUND.

TABLE: STUDENT_ITEMS (PK sequence: SEQ_STUDENT_ITEMS)		
Column	Type	Constraint / Notes
ITEM_ID	NUMBER	PRIMARY KEY — auto from SEQ_STUDENT_ITEMS
STUDENT_ID	NUMBER	FK → USERS.USER_ID (student only)
ITEM_NAME	VARCHAR2(100)	e.g. Laptop, Bag
CATEGORY	VARCHAR2(50)	e.g. Electronics, Clothing
BRAND	VARCHAR2(50)	nullable
MODEL	VARCHAR2(50)	nullable
SERIAL_NUMBER	VARCHAR2(100)	nullable — loaded into BST at login
STATUS	VARCHAR2(20)	REGISTERED OUT_OF_CAMPUS LOST STOLEN
LOCATION_ID	NUMBER	FK → LOCATIONS.LOCATION_ID (nullable)

■ SecurityCheckWindow loads all serial numbers into a BST for $O(\log n)$ lookup.

TABLE: LOST_FOUND (PK sequence: SEQ_LOST_FOUND)		
Column	Type	Constraint / Notes
RECORD_ID	NUMBER	PRIMARY KEY — auto from SEQ_LOST_FOUND
REPORTED_BY	NUMBER	FK → USERS.USER_ID
RECORD_TYPE	VARCHAR2(10)	LOST FOUND
ITEM_DESCRIPTION	VARCHAR2(500)	text description of the item
FOUND_LOCATION	NUMBER	FK → LOCATIONS.LOCATION_ID
REPORTED_AT	DATE	DEFAULT SYSDATE

TABLE: AUDIT_LOG (PK sequence: —)		
Column	Type	Constraint / Notes
LOG_ID	NUMBER	PRIMARY KEY
USER_ID	NUMBER	FK → USERS.USER_ID
ACTION	VARCHAR2(100)	e.g. LOGIN, DELETE_ASSET
TABLE_NAME	VARCHAR2(50)	affected table
RECORD_ID	NUMBER	affected row PK
LOG_TIME	DATE	DEFAULT SYSDATE

■ Auto-populated by triggers or application code for audit trail.

4. Stored Procedures

Procedure	Parameters	Action
SP_REPORT_ISSUE	p_asset_id, p_reported_by, p_description, p_priority	Inserts a new row into MAINTENANCE_LOG with STATUS='PENDING'
SP_RESOLVE_MAINTENANCE	p_maintenance_id, p_resolved_by, p_note	Sets STATUS='RESOLVED' on the given MAINTENANCE_ID, logs

5. Sequences (Oracle Auto-Increment)

Sequence Name	Used In Table	Oracle SQL to create
SEQ_USERS	USERS	CREATE SEQUENCE SEQ_USERS START WITH 1 INCREMENT BY 1;
SEQ_ASSETS	ASSETS	CREATE SEQUENCE SEQ_ASSETS START WITH 1 INCREMENT BY 1;
SEQ_STUDENT_ITEMS	STUDENT_ITEMS	CREATE SEQUENCE SEQ_STUDENT_ITEMS START WITH 1 INCREMENT BY 1;
SEQ_LOST_FOUND	LOST_FOUND	CREATE SEQUENCE SEQ_LOST_FOUND START WITH 1 INCREMENT BY 1;

6. How DSA Algorithms Connect to the Database

Algorithm	Chapter	Table Used	How it connects
Bubble Sort	Ch. 2	ASSETS	Sorts fetched rows by ASSET_ID or PURCHASE_VALUE in memory ($O(n^2)$)
Selection Sort	Ch. 2	ASSETS	Sorts rows by ASSET_NAME or LOCATION in memory
Linked List	Ch. 3	All tables	Action log records every DB INSERT/UPDATE/DELETE as a node
Stack (Undo/Redo)	Ch. 4	ASSETS	Before any DB write, snapshot pushed to undo stack
Recursion	Ch. 5	STUDENT_ITEMS	BST insert/search/height helpers are recursive
BST	Ch. 6	STUDENT_ITEMS	All SERIAL_NUMBERS loaded from DB into BST for $O(\log n)$ search
Graph BFS/DFS	Ch. 7	— (in-memory)	Campus building nodes modelled after real DBU locations
Merge Sort	Ch. 8	ASSETS	Sorts rows by PURCHASE_VALUE or ASSET_NAME $O(n \log n)$
Greedy	Ch. 9	MAINTENANCE_LOG	Reads PRIORITY column, scores, re-orders rows by score desc

7. Useful Oracle Commands for Testing

Connect to the schema

```
sqlplus campus_admin/your_password@localhost:1522/xe
```

Check all tables

```
SELECT table_name FROM user_tables ORDER BY table_name;
```

Check row counts

```
SELECT COUNT(*) FROM USERS;
```

```
SELECT COUNT(*) FROM ASSETS;
```

```
SELECT COUNT(*) FROM MAINTENANCE_LOG;
```

```
SELECT COUNT(*) FROM STUDENT_ITEMS;
```

```
SELECT COUNT(*) FROM LOST_FOUND;
```

```
SELECT COUNT(*) FROM LOCATIONS;  
SELECT COUNT(*) FROM AUDIT_LOG;
```

Reset sequences (re-seed to 1)

```
DROP SEQUENCE SEQ_USERS;  
CREATE SEQUENCE SEQ_USERS START WITH 1 INCREMENT BY 1;  
DROP SEQUENCE SEQ_ASSETS;  
CREATE SEQUENCE SEQ_ASSETS START WITH 1 INCREMENT BY 1;  
DROP SEQUENCE SEQ_STUDENT_ITEMS;  
CREATE SEQUENCE SEQ_STUDENT_ITEMS START WITH 1 INCREMENT BY 1;  
DROP SEQUENCE SEQ_LOST_FOUND;  
CREATE SEQUENCE SEQ_LOST_FOUND START WITH 1 INCREMENT BY 1;
```

Truncate all data (caution!)

```
TRUNCATE TABLE AUDIT_LOG;  
TRUNCATE TABLE LOST_FOUND;  
TRUNCATE TABLE STUDENT_ITEMS;  
TRUNCATE TABLE MAINTENANCE_LOG;  
TRUNCATE TABLE ASSETS;  
TRUNCATE TABLE USERS;  
TRUNCATE TABLE LOCATIONS;
```

Truncate order respects FK constraints: child tables first, parent tables last.

Test stored procedures

```
BEGIN SP_REPORT_ISSUE(1, 1, 'Projector broken', 'HIGH'); END;  
BEGIN SP_RESOLVE_MAINTENANCE(1, 1, 'Replaced lamp'); END;
```

View maintenance log with joins

```
SELECT m.MAINTENANCE_ID, a.ASSET_NAME, u.FULL_NAME, m.STATUS, m.PRIORITY  
FROM MAINTENANCE_LOG m  
JOIN ASSETS a ON m.ASSET_ID = a.ASSET_ID  
JOIN USERS u ON m.REPORTED_BY = u.USER_ID  
ORDER BY m.MAINTENANCE_ID;
```

View student items with owner name

```
SELECT si.ITEM_ID, u.FULL_NAME, si.ITEM_NAME, si.STATUS, si.SERIAL_NUMBER  
FROM STUDENT_ITEMS si  
JOIN USERS u ON si.STUDENT_ID = u.USER_ID  
ORDER BY si.ITEM_ID;
```